

Session 6:

```
backend > app > routers > products.py > {} APIRouter
41
42 @router.get("/products", response_model=List[ProductDetailResponse])
43 def get_products(
44     request: Request,
45     db: Session = Depends(get_db),
46     skip: int = Query(0, ge=0),
47     limit: int = Query(20, ge=1, le=1000),
48     category_slug: Optional[str] = None,
49     brand_slug: Optional[str] = None,
50     search: Optional[str] = None
51 ):
52     """
53     Lấy danh sách sản phẩm. Có thể phân trang, lọc theo category, brand, tìm kiếm,
54     và các thông số kỹ thuật động (specifications).
55     """
56     query = db.query(Product).filter(Product.is_published == True)
57
58     # Cần join trước nếu có filter theo tag
59     if category_slug or brand_slug:
60         query = query.outerjoin(Category).outerjoin(Brand)
61
62     # Filter by category exactly by slug (supports comma-separated multiple slugs)
63     if category_slug:
64         slugs = [s.strip().lower() for s in category_slug.split(',')]
65         if len(slugs) == 1:
66             query = query.filter(func.lower(Category.slug) == slugs[0])
67         else:
68             query = query.filter(func.lower(Category.slug).in_(slugs))
69
70     # Filter by brand exactly by slug or name
71     if brand_slug:
72         query = query.filter(
```

Hình 1: Phương thức GET lấy danh sách sản phẩm có phân trang, lọc, tìm kiếm

```

backend > app > routers > admin_products.py > {} APIRouter
46
47 @router.post("", response_model=dict, status_code=status.HTTP_201_CREATED)
48 def create_product(
49     product_in: ProductCreateUpdate,
50     db: Session = Depends(get_db),
51     admin: User = Depends(get_current_admin)
52 ):
53     # Tạo slug tự động từ tên (giản lược)
54     def slugify(text: str) -> str:
55         text = unicodedata.normalize('NFKD', text).encode('ascii', 'ignore').decode('utf-8')
56         text = re.sub(r'^\w\s-', '', text).strip().lower()
57         return re.sub(r'[-\s]+', '-', text)
58
59     slug = slugify(product_in.name)
60     existing = db.query(Product).filter(Product.slug == slug).first()
61     if existing:
62         slug = f"{slug}-{str(uuid.uuid4())[:8]}"
63
64     # Find or create Category - tìm bằng NAME trước (vì form gửi tên), rồi mới tìm bằng slug
65     category_id = None
66     if product_in.category:
67         cat = db.query(Category).filter(Category.name == product_in.category).first()
68         if not cat:
69             cat_slug = slugify(product_in.category)
70             cat = db.query(Category).filter(Category.slug == cat_slug).first()
71         if not cat:
72             cat_slug = slugify(product_in.category)
73             cat = Category(id=str(uuid.uuid4()), name=product_in.category, slug=cat_slug)
74             db.add(cat)
75             db.flush()
76         category_id = cat.id
77
78     # Find or create Brand

```

Hình 2: Phương thức POST tạo thông tin sản phẩm mới

```

151 @router.put("/{product_id}", response_model=dict)
152 def update_product(
153     product_id: str,
154     product_in: ProductCreateUpdate,
155     db: Session = Depends(get_db),
156     admin: User = Depends(get_current_admin)
157 ):
158     product = db.query(Product).filter(Product.id == product_id).first()
159     if not product:
160         raise HTTPException(status_code=404, detail="Sản phẩm không tồn tại")
161
162     product.name = product_in.name
163     product.description = product_in.description
164     product.specifications = product_in.specifications or {}
165     product.is_published = product_in.is_published
166
167     # Update Category
168     if product_in.category:
169         def slugify(text: str) -> str:
170             text = unicodedata.normalize('NFKD', text).encode('ascii', 'ignore').decode('utf-8')
171             text = re.sub(r'^\w\s-', '', text).strip().lower()
172             return re.sub(r'[-\s]+', '-', text)
173
174         cat = db.query(Category).filter(Category.name == product_in.category).first()
175         if not cat:
176             cat_slug = slugify(product_in.category)
177             cat = db.query(Category).filter(Category.slug == cat_slug).first()
178         if cat:
179             product.category_id = cat.id
180
181     # Update Brand

```

Hình 3: Phương thức PUT cập nhật thông tin sản phẩm

DELETE	/api/addresses/{address_id}	Delete Address	🔒	▼
PUT	/api/addresses/{address_id}/default	Set Default Address	🔒	▼
shipping ^				
POST	/api/shipping/calculate	Calculate Shipping		▼
GET	/api/shipping/ghn/provinces	Get Provinces		▼
GET	/api/shipping/ghn/districts	Get Districts	📄	▼
GET	/api/shipping/ghn/wards	Get Wards		▼
Admin Products ^				
POST	/api/admin/products/upload-image	Upload Admin Image	🔒	▼
POST	/api/admin/products	Create Product	🔒	▼
GET	/api/admin/products/{product_id}	Get Product Detail	🔒	▼
PUT	/api/admin/products/{product_id}	Update Product	🔒	▼
DELETE	/api/admin/products/{product_id}	Delete Product	🔒	▼
Admin Orders ^				
GET	/api/admin/orders	Get All Orders	🔒	▼
GET	/api/admin/orders/{order_id}	Get Order Detail	🔒	▼

Hình 4: Giao diện Swagger UI để request